

Bab 3 — Docker Network, Volume, Bind Mount, tmpfs, dan Compose

Nama	Arsyita Devanaya Arianto
NIM	3126640046
Kelas	D4 RPL IT B
Tanggal Pelaksanaan	23 September 2026

1. Tujuan dan Ruang Lingkup

Praktikum Bab 3 bertujuan untuk:

1. Membuat user-defined bridge network dan membuktikan name resolution antar container.
2. Membedakan volume, bind mount, dan tmpfs dari sisi persistensi, portabilitas, dan keamanan.
3. Menulis file Compose untuk aplikasi multi-container yang memiliki service, network, volume, dan healthcheck.
4. Mengelola lifecycle aplikasi dengan `docker compose up`, `ps`, `logs`, `stop`, `start`, `down`, dan `down -v`.

Ruang lingkup bab ini mencakup eksperimen user-defined bridge network, pembuatan dan restore volume (`backup data-vol`), serta stack Compose Nginx–Flask–PostgreSQL dengan segmentasi network dan healthcheck.

2. Landasan Teori

2.1 Network sebagai Graf Keterjangkauan

Jaringan container membentuk graf keterjangkauan antarkomponen, bukan sekadar pemberi alamat IP. Network frontend menghubungkan reverse proxy dengan aplikasi, sedangkan network backend menghubungkan aplikasi dengan database; database tidak perlu bergabung ke frontend dan

umumnya tidak perlu memublikasikan port ke host. Pemodelan ini mengurangi jalur serangan sekaligus memperjelas dependensi komunikasi.

2.2 User-Defined Bridge, DNS, dan Identitas Service

User-defined bridge menyediakan isolasi yang lebih baik dan resolusi DNS otomatis berbasis nama container atau nama service. Identitas logis (misalnya `db:5432` atau `app:5000`) lebih stabil daripada alamat IP yang dapat berubah ketika container dibuat ulang. Nama service bukan mekanisme autentikasi: DNS internal membantu discovery, tetapi tidak membuktikan identitas cryptographic peer dan tidak mengenkripsi lalu lintas. Network segmentation juga tidak menggantikan authorization; service yang terhubung ke dua zona tetap menjadi jalur sah yang dapat digunakan penyerang apabila service dikompromikan.

2.3 Port Publishing, NAT, dan Firewall Host

Opsi `-p HOST_PORT:CONTAINER_PORT` membuat aturan pada host agar traffic diteruskan menuju port container; komunikasi service-to-service menggunakan port container, bukan host port. Publikasi tanpa alamat bind (`-p 8080:80`) dapat mengikat seluruh interface host, sedangkan `-p 127.0.0.1:8080:80` membatasinya ke loopback. Perubahan manual pada iptables/nftables tanpa memahami chain Docker dapat menghasilkan policy yang tampak benar tetapi tidak berlaku pada traffic container; validasi harus dilakukan dari sumber jaringan yang relevan.

2.4 Lifecycle Data dan Pemilihan Mount

Mekanisme	Persistensi	Ketergantungan Host	Kasus Penggunaan	Risiko Utama
Writable layer	Hilang saat container dihapus	Rendah	Data sementara kecil	Sulit di-backup; CoW overhead
Named volume	Melampaui lifecycle container	Rendah-sedang	Database dan data aplikasi	Salah hapus; backup/restore belum otomatis
Bind mount	Mengikuti file host	Tinggi	Source, konfigurasi, artefak dev	Container dapat mengubah host; path tidak portabel

Mekanisme	Persistensi	Ketergantungan Host	Kasus Penggunaan	Risiko Utama
tmpfs	Hilang saat stop/restart	Linux dan memori host	Cache atau data temporer sensitif	Konsumsi RAM; bukan penyimpanan tahan lama
Compose secret	Selama deployment; sumber eksternal tetap ada	Bergantung sumber file/env	Password, token, certificate	Proteksi sumber dan permission tetap wajib

Persistent tidak sama dengan protected: volume dapat terhapus melalui `down -v` atau `prune`, mengalami korupsi, dan tidak otomatis memiliki backup. Bind mount memiliki akses tulis secara default sehingga opsi `:ro` harus dipakai bila write tidak diperlukan; mount ke direktori container yang telah berisi file akan menutupi isi tersebut selama mount aktif. tmpfs menyimpan data di memori host, cocok untuk cache atau data temporer sensitif, tetapi menghabiskan RAM dan tidak untuk data yang harus bertahan.

2.5 Compose sebagai Model Aplikasi Deklaratif

Docker Compose menyatakan model aplikasi multi-container dalam YAML dengan elemen `services`, `networks`, `volumes`, `configs`, dan `secrets`. Perintah `docker compose config` digunakan untuk memvalidasi model akhir setelah interpolation dan merge. Project name menjadi prefix network, volume, dan container. `up` merekonsiliasi keadaan aktual dengan model, `stop` menghentikan container tanpa menghapusnya, sedangkan `down` menghapus container dan network proyek; `down -v` bersifat destruktif karena turut menghapus volume. `depends_on` dengan `condition: service_healthy` menunggu dependency benar-benar siap, tidak sekadar dimulai.

2.6 Konfigurasi, Environment, dan Secret

Environment variable membantu memisahkan konfigurasi dari image, tetapi berpotensi terlihat melalui `inspect`, `process environment`, `log`, atau `crash report`; `.env` terutama merupakan sumber interpolation dan tidak otomatis aman untuk credential. Compose secret diberikan hanya pada service yang memerlukan dan tersedia sebagai file di `/run/secrets`. Password yang dipakai di laboratorium bersifat sintetis untuk lingkungan disposable; pada environment production-like, password tidak boleh

dikomit ke repository dan harus dikelola oleh secret manager atau file secret dengan permission terbatas.

2.7 Sintesis Konsep Inti

Tipe mount	Lokasi	Persisten	Kapan digunakan
Volume	Dikelola Docker di /var/lib/docker/volumes	Ya	Database, data aplikasi, artifact jangka panjang
Bind mount	Path host yang ditentukan user	Ya	Live development, file konfigurasi host
tmpfs	RAM host	Tidak	Cache sementara, secret lab, file sensitif sementara

3. Metodologi

Laboratorium menggunakan satu host yang menjalankan Docker Engine dan Compose v2 dengan direktori kerja `~/docker-lab/bab-3`. Setiap sub-langkah dijalankan berurutan dan diverifikasi sebelum lanjut. Perintah cleanup dijalankan setelah praktikum, terutama karena port yang sama dapat digunakan bab berikutnya. Kredensial database pada stack Compose menggunakan nilai sintetis yang di-redact pada laporan ini.

4. Pelaksanaan Praktikum

4.1 Pembuatan Direktori Bab 3

Membuat struktur direktori kerja proyek, yaitu `~/docker-lab/bab-3/app` (tempat aplikasi Flask) dan `~/docker-lab/bab-3/html` (tempat file statis Nginx), lalu pindah ke direktori tersebut sebagai lokasi utama praktikum.

```
mkdir -p ~/docker-lab/bab-3/app
mkdir -p ~/docker-lab/bab-3/html
cd ~/docker-lab/bab-3
```

```
ubuntu123@DESKTOP-7A8ST00:~$ mkdir -p ~/docker-lab/bab-3/app
ubuntu123@DESKTOP-7A8ST00:~$ mkdir -p ~/docker-lab/bab-3/html
ubuntu123@DESKTOP-7A8ST00:~$ cd ~/docker-lab/bab-3
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ ls
app  html
```

4.2 User-Defined Bridge Network

Membuat network lab-net tipe bridge dengan subnet 172.20.0.0/16 , menjalankan dua container nginx (server-a dan server-b) di network tersebut, lalu membuktikan *name resolution* antarkontainer dengan ping dari server-a ke server-b menggunakan nama container (bukan IP). Kedua container dihapus setelah pengujian.

```
docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
docker run -d --name server-b --network lab-net nginx:alpine
docker exec server-a ping -c 3 server-b
docker rm -f server-a server-b
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker network create --driver
bridge --subnet 172.20.0.0/16 lab-net
bd48ee0fd387f9ae6a59ac9d8ae73c446d9759f5a1a3c846426e6227335bfe2b
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker run -d --name server-a
--network lab-net nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
55afalecc21d: Pull complete
58c524ea09ce: Pull complete
850bf2dcecff: Pull complete
51900e10fb9c: Pull complete
8f924cf5086c: Pull complete
bc98d7675616: Pull complete
6636b9fc203c: Pull complete
af7dd138f459: Pull complete
a32eccfe9457: Download complete
10d8afe205cd: Download complete
Digest: sha256:72ba65eb42c10344912a84ff42408db7d34f2feb642204570ab8fc5ffd29f
1d3
Status: Downloaded newer image for nginx:alpine
8bf8fda98a7637b3764b4118424068fdd147f5519fd7b5e7662b9ef118af1db3
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker run -d --name server-b  
--network lab-net nginx:alpine  
d32493f6e2982b6ee9858122a375ba4b06d5f7ecdd3c16c391f49898fce6eab4  
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker exec server-a ping -c 3  
server-b  
PING server-b (172.20.0.3): 56 data bytes  
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.313 ms  
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.104 ms  
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.196 ms  
  
--- server-b ping statistics ---  
3 packets transmitted, 3 packets received, 0% packet loss  
round-trip min/avg/max = 0.104/0.204/0.313 ms  
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker rm -f server-a server-b  
server-a  
server-b  
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

4.2 Volume Backup dan Restore

Membuat named volume `data-vol` , menjalankan container `writer` yang menulis log waktu ke volume tersebut, lalu menghapus container untuk membuktikan data tetap ada di volume. Data kemudian dibaca ulang lewat container `cat` , dan dibackup ke host dalam bentuk `.tar.gz` melalui bind mount `$(pwd):/backup` .

```
docker volume create data-vol  
docker run -d --name writer -v data-vol:/app/data alpine:3.20 \  
sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"  
sleep 15  
docker rm -f writer  
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt  
docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 \  
tar czf /backup/data-vol-backup.tar.gz -C /source .
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker volume create data-vol  
data-vol  
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker run -d --name writer -v
data-vol:/app/data alpine:3.20 sh -c "while true; do date >> /app/data/lo
g.txt; sleep 5; done"
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
3a030ca3f633: Download complete
d4050d56ebf2: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b
6bc
Status: Downloaded newer image for alpine:3.20
f7d0d4f3782a3798b12c4d4a5de4e3e3723329c9a46965ca84ede991577719f3
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ sleep 15
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker rm -f writer
writer
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri Sep 11 16:03:17 UTC 2026
Fri Sep 11 16:03:22 UTC 2026
Fri Sep 11 16:03:27 UTC 2026
Fri Sep 11 16:03:32 UTC 2026
Fri Sep 11 16:03:37 UTC 2026
Fri Sep 11 16:03:42 UTC 2026
Fri Sep 11 16:03:47 UTC 2026
Fri Sep 11 16:03:52 UTC 2026
Fri Sep 11 16:03:57 UTC 2026
Fri Sep 11 16:04:02 UTC 2026
Fri Sep 11 16:04:07 UTC 2026
Fri Sep 11 16:04:12 UTC 2026
Fri Sep 11 16:04:17 UTC 2026
Fri Sep 11 16:04:22 UTC 2026
Fri Sep 11 16:04:27 UTC 2026
Fri Sep 11 16:04:32 UTC 2026
Fri Sep 11 16:04:37 UTC 2026
Fri Sep 11 16:04:42 UTC 2026
Fri Sep 11 16:04:47 UTC 2026
Fri Sep 11 16:04:52 UTC 2026
Fri Sep 11 16:04:57 UTC 2026
Fri Sep 11 16:05:02 UTC 2026
Fri Sep 11 16:05:07 UTC 2026
Fri Sep 11 16:05:12 UTC 2026
Fri Sep 11 16:05:17 UTC 2026
Fri Sep 11 16:05:22 UTC 2026
Fri Sep 11 16:05:27 UTC 2026
Fri Sep 11 16:05:32 UTC 2026
Fri Sep 11 16:05:37 UTC 2026
Fri Sep 11 16:05:42 UTC 2026
Fri Sep 11 16:05:47 UTC 2026
Fri Sep 11 16:05:52 UTC 2026
Fri Sep 11 16:05:57 UTC 2026
Fri Sep 11 16:06:02 UTC 2026
Fri Sep 11 16:06:07 UTC 2026
Fri Sep 11 16:06:12 UTC 2026
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

4.3 Compose Multi-Container Nginx–Flask–PostgreSQL

Arsitektur stack: web (Nginx) berada hanya di network frontend , app (Flask) menghubungkan frontend dan backend , sedangkan db (PostgreSQL) hanya di backend sehingga database tidak terekspos ke internet. Data DB disimpan pada volume pg-data dan Compose menunggu db healthy (pg_isready) sebelum app dijalankan.

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]
  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: <REDACTED>
    networks: [frontend, backend]
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: <REDACTED>
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 5s
      timeout: 5s
      retries: 5
volumes:
  pg-data:
networks:
  frontend:
  backend:
```

1. Menulis compose.yaml

Tulis file `compose.yaml` sebagai definisi deklaratif stack, berisi 3 service (`web` = Nginx, `app` = Flask, `db` = PostgreSQL), 2 network (`frontend` , `backend`), dan 1 named volume (`pg-data`) beserta healthcheck pada service `db` .

```
nano compose.yaml
```

```
services:
  web:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - frontend
    depends_on:
      - app
```

```
  app:
    build:
      context: ./app
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks:
      - frontend
      - backend
    depends_on:
      db:
        condition: service_healthy
```

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U labuser -d labdb
      interval: 5s
      timeout: 5s
      retries: 5
```

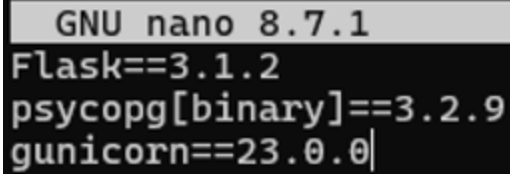
```
volumes:
  pg-data:
```

```
networks:
  frontend:
  backend:
```

2. Membuat aplikasi Flask

Buat `requirements.txt` (daftar dependensi Python) dan `app.py` (kode aplikasi yang menyambung ke DB melalui `DB_HOST: db` serta menyediakan endpoint `/health` untuk mengecek koneksi database).

```
nano app/requirements.txt
```

A screenshot of a terminal window with a black background. At the top, a grey header bar displays 'GNU nano 8.7.1'. Below the header, the text 'Flask==3.1.2', 'psycopg[binary]==3.2.9', and 'gunicorn==23.0.0' is visible, with a cursor at the end of the last line.

```
GNU nano 8.7.1
Flask==3.1.2
psycopg[binary]==3.2.9
gunicorn==23.0.0|
```

```
nano app/app.py
```

```

GNU nano 8.7.1                                app/app.py *
import os

import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)

def database_connection():
    return psycopg2.connect(
        host=os.environ["DB_HOST"],
        port=os.getenv("DB_PORT", "5432"),
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASS"],
    )

@app.get("/")
def index():
    try:
        with database_connection() as connection:
            with connection.cursor() as cursor:
                cursor.execute("SELECT version();")
                database_version = cursor.fetchone()[0]

        return jsonify(
            status="ok",
            message="Nginx, Flask, dan PostgreSQL berhasil terhubung",
            database=database_version,
        )
    except Exception as error:
        return jsonify(
            status="error",
            message="Koneksi database gagal",
            detail=str(error),
        ), 500

@app.get("/health")
def health():
    try:
        with database_connection() as connection:
            with connection.cursor() as cursor:
                cursor.execute("SELECT 1;")
                cursor.fetchone()

        return jsonify(status="healthy"), 200
    except Exception:
        return jsonify(status="unhealthy"), 503

```

3. Membuat Dockerfile aplikasi

Tulis `Dockerfile` untuk membangun image aplikasi Flask. Pipeline ini memakai base image `python:3.11-slim` (Debian) sebagai solusi kegagalan `apk add` pada Alpine, sehingga proses build

lebih cepat dan sukses.

```
nano app/Dockerfile
```

```
GNU nano 8.7.1 app/Dockerfile *
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --system --uid 10001 appuser
USER appuser

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

4. Membuat konfigurasi Nginx

Siapkan `nginx.conf` agar Nginx (service web) berperan sebagai reverse proxy yang menerima request dari host pada port 8080 lalu meneruskannya ke service `app` , sekaligus melayani file statis dari `html/` .

```
nano nginx.conf
```

```
GNU nano 8.7.1 nginx.conf *
upstream flask_backend {
    server app:5000;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://flask_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }

    location = /static.html {
        root /usr/share/nginx/html;
    }
}
```

5. Membuat halaman statis opsional

Buat `html/index.html` sebagai konten statis yang dilayani langsung oleh Nginx melalui bind mount read-only (`./html:/usr/share/nginx/html:ro`).

```
nano html/index.html
```

```
GNU nano 8.7.1                               html/index.h
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Laboratorium Docker Compose</title>
</head>
<body>
  <h1>Docker Compose Bab 3</h1>
  <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>|
```

6. Memvalidasi Compose

`docker compose config` menampilkan model komposisi akhir (hasil interpolation dan merge) sekaligus memvalidasi kebenaran YAML; `--services` hanya mendaftarkan nama-nama service yang terdefinisi.

```
docker compose config
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose config
```

```
name: bab-3
```

```
services:
```

```
  app:
```

```
    build:
```

```
      context: /home/ubuntu123/docker-lab/bab-3/app
```

```
      dockerfile: Dockerfile
```

```
    depends_on:
```

```
      db:
```

```
        condition: service_healthy
```

```
        required: true
```

```
    environment:
```

```
      DB_HOST: db
```

```
      DB_NAME: labdb
```

```
      DB_PASS: labpass123
```

```
      DB_PORT: "5432"
```

```
      DB_USER: labuser
```

```
    networks:
```

```
      backend: null
```

```
      frontend: null
```

```
  db:
```

```
    environment:
```

```
      POSTGRES_DB: labdb
```

```
      POSTGRES_PASSWORD: labpass123
```

```
      POSTGRES_USER: labuser
```

```
    healthcheck:
```

```
      test:
```

```
        - CMD-SHELL
```

```
        - pg_isready -U labuser -d labdb
```

```
      timeout: 5s
```

```
      interval: 5s
```

```
      retries: 5
```

```
    image: postgres:16-alpine
```

```
    networks:
```

```
      backend: null
```

```
    volumes:
```

```
      - type: volume
```

```
        source: pg-data
```

```
        target: /var/lib/postgresql/data
```

```
        volume: {}
```

```
  web:
```

```
    depends_on:
```

```
      app:
```

```
        condition: service_started
```

```
        required: true
```

```
    image: nginx:alpine
```

```
    networks:
```

```
      frontend: null
```

```
    ports:
```

```
      - mode: ingress
```

```
        host_ip: 127.0.0.1
```

```
        target: 80
```

```
        published: "8080"
```

```
        protocol: tcp
```

```
    volumes:
```

```
      - type: bind
```

```
        source: /home/ubuntu123/docker-lab/bab-3/html
```

```
        target: /usr/share/nginx/html
```

```
        read_only: true
```

```
        bind: {}
```

```
- type: bind
  source: /home/ubuntu123/docker-lab/bab-3/nginx.conf
  target: /etc/nginx/conf.d/default.conf
  read_only: true
  bind: {}
networks:
  backend:
    name: bab-3_backend
  frontend:
    name: bab-3_frontend
volumes:
  pg-data:
    name: bab-3_pg-data
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
docker compose config --services
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose config --services
db
app
web
```

7. Menjalankan stack

`docker compose up -d --build` membangun image yang diperlukan lalu menjalankan semua container di background sesuai model Compose; `docker compose ps` digunakan untuk mengecek status dan port tiap service.

```
docker compose up -d --build
```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose up -d --build
[+] Building 79.4s (14/14) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 508B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 307B 0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 6.6s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9 10.0s
=> => resolve docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a 0.0s
=> => sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca 249B / 249B 0.6s
=> => sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf 12.12MB / 12.12MB 3.9s
=> => sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70 1.29MB / 1.29MB 3.2s
=> => sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c 29.83MB / 29.83MB 9.1s
=> => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8 0.5s
=> => extracting sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70 0.1s
=> => extracting sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf0 0.2s
=> => extracting sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 1.40kB 0.0s
=> [2/6] WORKDIR /app 0.1s
=> [3/6] COPY requirements.txt . 0.0s
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt 60.9s
=> [5/6] COPY app.py . 0.0s
=> [6/6] RUN useradd --system --uid 10001 appuser 0.5s
=> exporting to image 1.1s
=> => exporting layers 0.8s
=> => exporting manifest sha256:499787b3968cc61a4c943ee404f7d8fb30739eb3006134e0540e26a783bdf3 0.0s
=> => exporting config sha256:2d2d9549f66b674a3a9c4ce30e57f48f050a1e8f81ea9246adc96ce65f20c137 0.0s
=> => exporting attestation manifest sha256:bd73fe371755eb10a1a7c006bc129941ac8eab41fdc9c54386 0.0s
=> => exporting manifest list sha256:81db9a01f4c14fa998ddba76d1f03144e98defff94164df99bebc12d3 0.0s
=> => naming to docker.io/library/bab-3-app:latest 0.0s
=> => unpacking to docker.io/library/bab-3-app:latest 0.2s
=> resolving provenance for metadata file 0.0s
[+] up 4/4
✓Image bab-3-app Built 79.7s
✓Container bab-3-app-1 Started 7.1s
✓Container bab-3-web-1 Started 7.6s
✓Container bab-3-db-1 Healthy 6.0s

```

docker compose ps

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
bab-3-app-1	bab-3-app	"gunicorn --bind 0.0..."	app	About a minute ago	Up About a minute
bab-3-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	10 days ago	Up About a minute (healthy)
bab-3-web-1	nginx:alpine	"/docker-entrypoint..."	web	About a minute ago	Up About a minute

8. Menguji aplikasi

Uji alur reverse proxy: `curl http://localhost:8080/` meminta halaman utama dari Nginx, `/health` memverifikasi koneksi Flask ke PostgreSQL, dan `/static.html` memastikan file statis terlayani.

```
curl http://localhost:8080/
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ curl http://localhost:8080/
{"database":"PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit",
,"message":"Nginx, Flask, dan PostgreSQL berhasil terhubung","status":"ok"}
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ |
```

```
{
  database: "PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0,
  64-bit",
  message: "Nginx, Flask, dan PostgreSQL berhasil terhubung",
  status: "ok"
}
```

```
curl -i http://localhost:8080/health
```

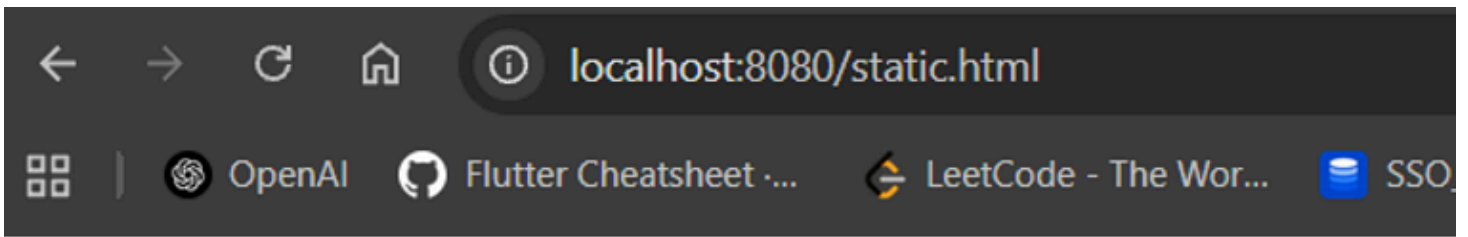
```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ curl -i http://localhost:8080/health
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Tue, 22 Sep 2026 15:46:46 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status":"healthy"}
```

```
curl http://localhost:8080/static.html
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ curl -i http://localhost:8080/static.html
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Tue, 22 Sep 2026 17:36:50 GMT
Content-Type: text/html
Content-Length: 227
Last-Modified: Tue, 22 Sep 2026 17:36:31 GMT
Connection: keep-alive
ETag: "6ab2bc9f-e3"
Accept-Ranges: bytes

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Laboratorium Docker Compose</title>
</head>
<body>
  <h1>Docker Compose Bab 3</h1>
  <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>
```



Docker Compose Bab 3

Bind mount Nginx berhasil digunakan.

9. Memeriksa log

`logs --tail 100` menampilkan 100 baris log terakhir seluruh service; `logs -f` mengikuti log secara real-time; `logs <service>` untuk melihat log per service (app, db, atau web).

```
docker compose logs --tail 100
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose logs --tail 100
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configurat
web-1 | ion
web-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-1 | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-on
web-1 | ly file system?)
web-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
```

```
docker compose logs -f
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose logs -f
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configurat
web-1 | ion
db-1 | The files belonging to this database system will be owned by user "postgres".
db-1 | This user must also own the server process.
db-1 |
db-1 | The database cluster will be initialized with locale "en_US.utf8".
```

```
docker compose logs app
```

```
docker compose logs db
```

```
docker compose logs web
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose logs app
app-1 | [2026-09-22 15:43:39 +0000] [1] [INFO] Starting gunicorn 23.0.0
app-1 | [2026-09-22 15:43:39 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
app-1 | [2026-09-22 15:43:39 +0000] [1] [INFO] Using worker: sync
app-1 | [2026-09-22 15:43:39 +0000] [7] [INFO] Booting worker with pid: 7
app-1 | [2026-09-22 15:43:39 +0000] [8] [INFO] Booting worker with pid: 8
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose logs db
db-1 | The files belonging to this database system will be owned by user "postgres".
db-1 | This user must also own the server process.
db-1 |
db-1 | The database cluster will be initialized with locale "en_US.utf8".
db-1 | The default database encoding has accordingly been set to "UTF8".
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-3$ docker compose logs web
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
web-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
web-1 | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system?)
web-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
web-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
web-1 | 2026/09/22 15:43:39 [notice] 1#1: using the "epoll" event method
```

10. Menghentikan lingkungan

`docker compose down` menghentikan dan menghapus container serta network proyek; `down -v` bersifat destruktif karena sekaligus menghapus named volume `pg-data` (data PostgreSQL ikut hilang).

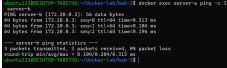
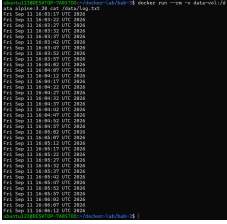
```
docker compose down
```

```
docker compose down -v -- menghapus container, network, sekaligus volume PostgreSQL
```

11. Struktur Direktori

Menampilkan susunan file proyek final sebagai dokumentasi: `compose.yaml`, direktori `app/` (Flask + Dockerfile), `html/` (file statis), dan `nginx.conf`.

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab$ tree
.
├── bab-2
│   └── custom-web
│       ├── Dockerfile
│       └── index.html
├── bab-3
│   ├── app
│   │   ├── Dockerfile
│   │   ├── app.py
│   │   └── requirements.txt
│   ├── compose.yaml
│   ├── html
│   │   └── index.html
│   └── nginx.conf
```

Perintah	Hasil / Catatan	Status
docker network create ... lab-net		PASS
docker exec server-a ping -c 3 server-b		PASS
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt		PASS
docker compose ps		PASS
curl -v http://localhost:8080 (API health)		PASS
docker compose logs --tail 100		PASS

5. Verifikasi

Skenario Pengujian	Hasil
Container di user-defined bridge dapat saling resolve menggunakan nama	PASS
Data di named volume tetap ada setelah container dihapus	PASS
Bind mount menunjukkan perubahan file host tanpa rebuild image	PASS

Skenario Pengujian	Hasil
tmpfs kehilangan data setelah container restart	PASS
Compose stack web-app-db berjalan dan API health menampilkan koneksi database	PASS

6. Analisis Wajib

6.1 Jelaskan satu masalah yang muncul dan cara Anda mendiagnosisnya.

- Masalah: Saat mengakses `curl http://localhost:8080/static.html`, halaman tidak muncul (respon bukan halaman yang diharapkan). Nginx hanya melayani file yang benar-benar ada di direktori `html/` yang di-bind mount, sedangkan `static.html` belum dibuat di folder tersebut.
- Diagnosis: Dengan `curl -i http://localhost:8080/static.html` terlihat kode status respon dari Nginx (404/error), dan pemeriksaan pada direktori `html/` menunjukkan bahwa file `static.html` memang tidak ada, sehingga Nginx tidak dapat menemukan file yang diminta untuk disajikan.
- Solusi: Membuat file `static.html` di dalam folder `html/`. Karena direktori tersebut di-bind mount (`./html:/usr/share/nginx/html:ro`), file langsung dapat diakses tanpa rebuild image, dan `curl http://localhost:8080/static.html` kemudian berhasil menampilkan halaman tersebut.

6.2 Jelaskan risiko keamanan atau operasional yang relevan pada bab ini

- Hardcoded Credentials: Password database ditulis plain text di `docker-compose.yml`, berisiko bocor ke version control.
- Privilege Root: Aplikasi berjalan sebagai root di dalam container, meningkatkan risiko keamanan jika terkena celah Remote Code Execution (RCE).
- Tanpa Limit Resources: Container belum diberi batasan CPU/RAM, berpotensi memicu Out-Of-Memory (OOM) atau Denial of Service (DoS).

6.3 Berikan rekomendasi perbaikan bila lab ini akan dibawa ke production-like environment.

- Gunakan Docker Secrets: Pindahkan password database dari Compose ke Docker Secrets atau Secret Manager.
- Jalankan Non-Root User: Tambahkan USER non-root pada Dockerfile dan batasi capabilities kernel.

- Terapkan Resource Limits: Tambahkan batasan CPU dan RAM pada tiap service di Compose (misal: memory: 512M).
- Aktifkan HTTPS/TLS: Gunakan SSL/TLS pada Nginx proxy untuk enkripsi lalu lintas data.

9. Kesimpulan

Praktikum Bab 3 berhasil mengimplementasikan arsitektur 3-tier terisolasi secara deklaratif menggunakan Docker Compose untuk mengordinasikan layanan Nginx, Python Flask, dan PostgreSQL. Melalui penggunaan user-defined bridge network (frontend dan backend), named volume (pg-data), serta konfigurasi healthcheck, sistem mampu menjaga keamanan isolasi database, menjamin persistensi data, dan memastikan dependensi antar-service berjalan secara stabil.